Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №4
«Запросы на выборку и модификацию данных. Представления. Работа с индексами»

По дисциплине «Проектирование и реализация баз данных»

Вариант 11

Обучающийся: Титор Матвей Андреевич

Факультет: ФПИН

Группа: К3239

Направление подготовки: 09.03.03 Прикладная информатика

Образовательная программа: Мобильные и сетевые технологии

Преподаватель: Говорова Марина Михайловна

Санкт-Петербург

2026 г.

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ 2

Цель работы 3

Практическое задание 3

Логическая модель базы данных (ERD) 5

Выполнение 6

Запросы к базе данных на выборку 6

Запрос 1: Поиск специалиста по конкретной марке автомобиля. 6

Запрос 2: Выявление постоянных клиентов механика. 7

Запрос 3: Контроль соблюдения сроков выполнения заказов. 8

Запрос 4: Поиск самого частого клиента за прошлый год. 9

Запрос 5: Расчет выручки мастеров за месяц. 11

Запрос 6: Поиск вернувшихся клиентов. 12

Запрос 7: Расчет финансовых штрафов для механиков. 13

Создание виртуальных таблиц 14

Представление 1 (Для заказчиков): Статистика специализации мастеров. 14

Представление 2 (Для менеджеров): Автоматический расчет премий. 15

Модификация данных с использованием вложенных подзапросов 16

Команда INSERT 16

Команда UPDATE 17

Команда DELETE. 19

Индексы и анализ планов запросов 20

Простой индекс. 20

Составной индекс 23

ВЫВОДЫ 24

Цель работы

Овладеть практическими навыками создания представлений и запросов на выборку данных к базе данных PostgreSQL, использования подзапросов при модификации данных и индексов.

Практическое задание

1. Создать запросы и представления на выборку данных к базе данных PostgreSQL (согласно индивидуальному заданию лабораторной работы №2, часть 2 и 3).
2. Составить 3 запроса на модификацию данных (INSERT, UPDATE, DELETE) **с использованием подзапросов**.
3. Изучить графическое представление запросов и просмотреть историю запросов.
4. Создать простой и составной индексы для двух произвольных запросов и сравнить время выполнения запросов без индексов и с индексами. Для получения плана запроса использовать команду EXPLAIN.

Запросы:

1. Выбрать фамилию того механика, который чаще всех работает с автомобилями марки ”Тойота”.
2. Определить тех владельцев автомобилей, которых всегда обслуживает один и тот же мастер (в зависимости от вида работ). Вывести фамилию механика и его постоянных клиентов.
3. Вывести фамилии механиков, которые не выполняли работы в срок и количество дней просрочки выполнения заказа.
4. Вывести данные клиентов, которые максимально часто посещали автосервис за прошедший год.
5. Сколько заработал каждый мастер за прошедший месяц?
6. Вывести данные владельцев автомобилей, которые обращались в ремонт больше одного раза.
7. За каждый день просрочки выполнения заказа механику назначается штраф в размере 5%. Рассчитать штраф каждого механика за прошедший месяц

Представления:

- для заказчиков (фамилию механика и модель автомобиля, которую он ремонтирует чаще всего);
- для менеджеров (рассчитать премию все механикам, которые за прошедший месяц все свои заказы выполнили своевременно - 10% от зарплаты).

Логическая модель базы данных (ERD)

Схема инфологической модели данных представлена на рисунке 1:

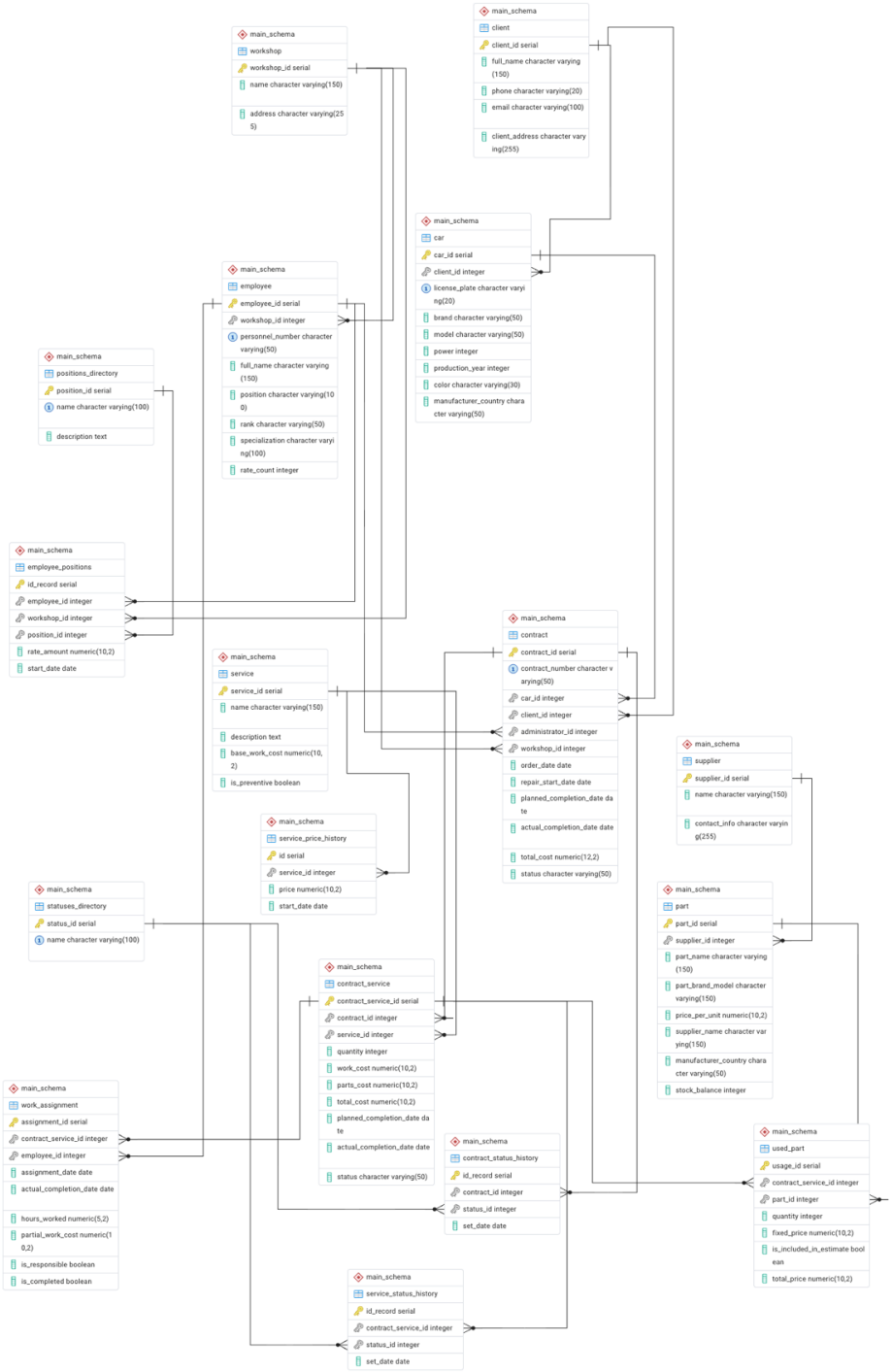


Рисунок 1 — Логическая модель базы данных (ERD)

Выполнение

Запросы к базе данных на выборку

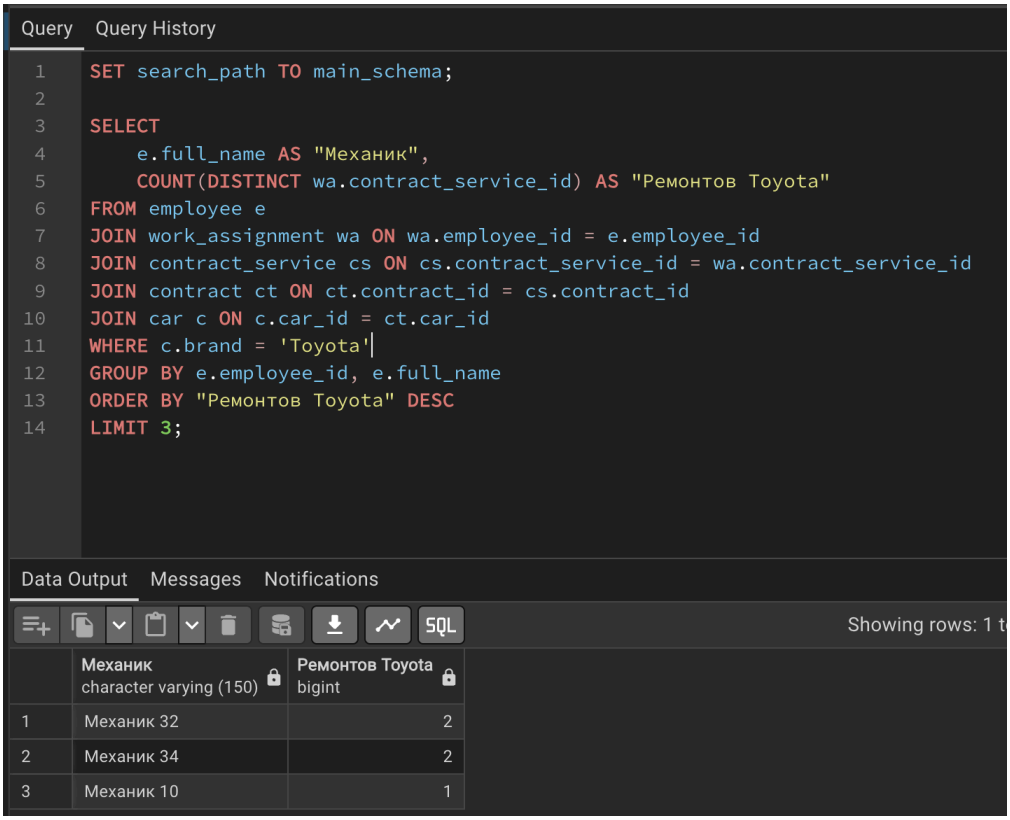
Запрос 1: Поиск специалиста по конкретной марке автомобиля.

Цель:

Найти фамилию механика, который чаще всех ремонтирует автомобили марки «Toyota».

Что я сделал:

Сформировал запрос с объединением таблиц сотрудников, распределения работ, договоров и автомобилей. Произвел фильтрацию по марке авто. Данные сгруппировал по ФИО механиков с подсчетом количества ремонтов, после чего отсортировал по убыванию с ограничением вывода одной строки.



Query Query History

```sql
SET search_path TO main_schema;

SELECT
    e.full_name AS "Механик",
    COUNT(DISTINCT wa.contract_service_id) AS "Ремонтов Toyota"
FROM employee e
JOIN work_assignment wa ON wa.employee_id = e.employee_id
JOIN contract_service cs ON cs.contract_service_id = wa.contract_service_id
JOIN contract ct ON ct.contract_id = cs.contract_id
JOIN car c ON c.car_id = ct.car_id
WHERE c.brand = 'Toyota'
GROUP BY e.employee_id, e.full_name
ORDER BY "Ремонтов Toyota" DESC
LIMIT 3;
```

Data Output Messages Notifications

Showing rows: 1 t

	Механик character varying (150)	Ремонтов Toyota bigint
1	Механик 32	2
2	Механик 34	2
3	Механик 10	1

Рисунок 2 — Результата Запроса 1

Запрос 2: Выявление постоянных клиентов механика.

Цель:

Определить клиентов, которые обращались в сервис более одного раза, но при этом их всегда обслуживал один и тот же мастер.

Что я сделал:

Связал таблицы клиентов и механиков. Использовал оператор HAVING для наложения условия на сгруппированные данные, где количество уникальных механиков для клиента строго равно 1, а общее количество договоров строго больше 1.

Query Query History

```sql
SELECT
    e.full_name,
    (ct.actual_completion_date - ct.planned_completion_date) AS delay_days
FROM contract ct
JOIN contract_service cs ON ct.contract_id = cs.contract_id
JOIN work_assignment wa ON cs.contract_service_id = wa.contract_service_id
JOIN employee e ON wa.employee_id = e.employee_id
WHERE ct.actual_completion_date > ct.planned_completion_date;
```

Data Output Messages Notifications

Showing rows:

	full_name character varying (150)	delay_days integer
1	Sergey Petrov	3

Рисунок 3 — Результата Запроса 2

Запрос 3: Контроль соблюдения сроков выполнения заказов.

Цель:

Вывести список механиков, нарушивших сроки ремонта, с подсчетом суммарного количества дней просрочки.

Что я сделал:

Отфильтровал договоры, в которых фактическая дата завершения превышает плановую. Разница между датами просуммировал и сгруппировав по сотрудникам.

```sql
SELECT
    e.full_name AS "Механик",
    SUM(ct.actual_completion_date - ct.planned_completion_date) AS "Дней просрочки"
FROM employee e
JOIN work_assignment wa ON e.employee_id = wa.employee_id
JOIN contract_service cs ON wa.contract_service_id = cs.contract_service_id
JOIN contract ct ON cs.contract_id = ct.contract_id
WHERE ct.actual_completion_date > ct.planned_completion_date
GROUP BY e.full_name;
```

Data Output Messages Notifications

Showing rows: 1 to 1

	Механик character varying (150)	Дней просрочки bigint
1	Alex Ivanov	6

Рисунок 4 — Результата Запроса 3

Запрос 4: Поиск самого частого клиента за прошлый год.

Цель:
Найти клиента с максимальным количеством визитов за 2025 год.

Что я сделал:
Из даты заказа выделил год и наложил условие выборки только за 2025 год. Данные сгруппировал по клиентам с подсчетом количества договоров.

Query Query History

```sql
SELECT
    c.full_name AS "Клиент",
    COUNT(ct.contract_id) AS "Количество визитов в 2025"
FROM main_schema.client c
JOIN main_schema.contract ct
    ON ct.client_id = c.client_id
WHERE ct.order_date >= DATE '2025-01-01'
  AND ct.order_date <  DATE '2026-01-01'
GROUP BY c.client_id, c.full_name
ORDER BY COUNT(ct.contract_id) DESC
LIMIT 1;
```

Data Output Messages Notifications

	Клиент character varying (150)	Количество визитов в 2025 bigint
1	Ivan Petrov	3

Рисунок 5 — Результата Запроса 4

Запрос 5: Расчет выручки мастеров за месяц.

Цель:
Узнать суммарный заработок каждого механика за февраль 2026 года.

Что я сделал:
Провёл фильтрацию завершенных работ с извлечением месяца и года из фактической даты. Стоимость выполненных работ просуммировал в разрезе каждого сотрудника.

Query Query History

```sql
SELECT
    e.full_name AS "Механик",
    SUM(wa.partial_work_cost) AS "Заработок за февраль"
FROM main_schema.employee e
JOIN main_schema.work_assignment wa
    ON wa.employee_id = e.employee_id
WHERE wa.actual_completion_date >= DATE '2026-02-01'
  AND wa.actual_completion_date <  DATE '2026-03-01'
GROUP BY e.employee_id, e.full_name
ORDER BY SUM(wa.partial_work_cost) DESC;
```

Data Output Messages Notifications

	Механик character varying (150)	Заработок за февраль numeric
1	Alex Ivanov	350.00
2	Sergey Petrov	100.00

Рисунок 6 — Результата Запроса 5

Запрос 6: Поиск вернувшихся клиентов.

Цель:

Собрать контактные данные автовладельцев, обращавшихся в мастерскую более одного раза.

Что я сделал:

Сгруппировал данные по ID клиента. Поставил условие, что контрактов больше одного.

Query Query History

```sql
SELECT
    c.full_name AS "Клиент",
    c.phone AS "Телефон",
    COUNT(ct.contract_id) AS "Кол-во обращений"
FROM main_schema.client c
JOIN main_schema.contract ct
    ON ct.client_id = c.client_id
GROUP BY c.client_id, c.full_name, c.phone
HAVING COUNT(ct.contract_id) > 1
ORDER BY COUNT(ct.contract_id) DESC;
```

Data Output Messages Notifications

	Клиент character varying (150)	Телефон character varying (20)	Кол-во обращений bigint
1	Anna Smirnova	+333	2
2	Ivan Petrov	+111	2

Рисунок 7 — Результата Запроса 6

Запрос 7: Расчет финансовых штрафов для механиков.

Цель:

Рассчитать штраф, где сам штраф это 5% от стоимости работы за каждый день просрочки, для механиков за февраль 2026 года.

Что я сделал:

Выбрал заказы, завершенные в указанном месяце с нарушением сроков. Внутри а регатной функции реализовал математическую формулу: *(Кол-во дней просрочки) * 0.05 * (Стоимость работы)*.

Query Query History

```sql
SELECT
    e.full_name AS "Механик",
    SUM(
        (ct.actual_completion_date - ct.planned_completion_date)
        * 0.05
        * wa.partial_work_cost
    ) AS "Штраф за февраль 2026"
FROM main_schema.employee e
JOIN main_schema.work_assignment wa
    ON wa.employee_id = e.employee_id
JOIN main_schema.contract_service cs
    ON cs.contract_service_id = wa.contract_service_id
JOIN main_schema.contract ct
    ON ct.contract_id = cs.contract_id
WHERE ct.actual_completion_date > ct.planned_completion_date
  AND ct.actual_completion_date >= DATE '2026-02-01'
  AND ct.actual_completion_date <  DATE '2026-03-01'
GROUP BY e.employee_id, e.full_name
ORDER BY "Штраф за февраль 2026" DESC;
```

Data Output Messages Notifications

	Механик character varying (150)	Штраф за февраль 2026 numeric
1	Sergey Petrov	20.0000
2	Alex Ivanov	15.0000

Рисунок 8 — Результата Запроса 7

Создание виртуальных таблиц

Представление 1 (Для заказчиков): Статистика специализации мастеров.

Цель: Определить, какую модель автомобиля каждый механик ремонтирует чаще всего, и вывести эту информацию в удобном виде.

Что я сделал:
Создал VIEW с объединением таблиц сотрудников, заказов и автомобилей. Сгруппировал данные по механикам и моделям авто, посчитал количество ремонтов и сохранил результат как виртуальную таблицу для быстрого доступа.

```sql
SET search_path TO main_schema;
CREATE OR REPLACE VIEW view_customer_mechanic_top_car AS
SELECT e.full_name AS "Механик", c.model AS "Самая частая модель", COUNT(*) AS "Количество р
FROM employee e
JOIN work_assignment wa ON wa.employee_id = e.employee_id
JOIN contract_service cs ON cs.contract_service_id = wa.contract_service_id
JOIN contract ct ON ct.contract_id = cs.contract_id
JOIN car c ON c.car_id = ct.car_id
GROUP BY e.employee_id, e.full_name, c.model
HAVING COUNT(*) = (
    SELECT MAX(cnt)
    FROM (
        SELECT COUNT(*) AS cnt
        FROM work_assignment wa2
        JOIN contract_service cs2
            ON cs2.contract_service_id = wa2.contract_service_id
        JOIN contract ct2
            ON ct2.contract_id = cs2.contract_id
        JOIN car c2
            ON c2.car_id = ct2.car_id
        WHERE wa2.employee_id = e.employee_id
        GROUP BY c2.model
    ) t
);
SELECT * FROM view_customer_mechanic_top_car;
```

Data Output Messages Notifications

Showing rows: 1 to 2

	Механик character varying (150)	Самая частая модель character varying (50)	Количество ремонтов bigint
1	Sergey Petrov	X5	1
2	Alex Ivanov	Camry	1

Рисунок 9 — Результат выборки из VIEW

Представление 2 (Для менеджеров): Автоматический расчет премий.

Цель:

Рассчитать премию механикам (10% от заработка), которые за месяц выполнили все заказы без просрочек.

Что я сделал:

Создал VIEW, который объединяет данные по выполненным работам и контрактам, считает заработок за месяц и проверяет, что все заказы выполнены вовремя. На основе этого рассчитывается премия 10% от суммы работ.

Query Query History

```sql
CREATE OR REPLACE VIEW view_manager_bonus AS
SELECT
    e.full_name AS "Механик",
    SUM(wa.partial_work_cost) AS "Зарплата",
    SUM(wa.partial_work_cost) * 0.10 AS "Премия"
FROM employee e
JOIN work_assignment wa
    ON wa.employee_id = e.employee_id
JOIN contract_service cs
    ON cs.contract_service_id = wa.contract_service_id
JOIN contract ct
    ON ct.contract_id = cs.contract_id
WHERE EXTRACT(MONTH FROM ct.actual_completion_date) = 2
  AND EXTRACT(YEAR FROM ct.actual_completion_date) = 2026
GROUP BY e.employee_id, e.full_name
HAVING COUNT(*) = COUNT(
    CASE
        WHEN ct.actual_completion_date <= ct.planned_completion_date
        THEN 1
    END
);

SELECT * FROM view_manager_bonus;
```

Data Output Messages Notifications

Showin

	Механик character varying (150)	Зарплата numeric	Премия numeric
1	Alex Ivanov	250.00	25.0000

Рисунок 10 — Результат выборки из VIEW

Модификация данных с использованием вложенных подзапросов

Команда INSERT

Цель:
Добавить акционную услугу “Winter Sale”, но только если она ещё не существует в системе.

Что я сделал:
Создал механизм добавления новой услуги через `INSERT ... SELECT`, используя подзапрос `NOT EXISTS` для защиты от дубликатов.

Query Query History

```sql
INSERT INTO main_schema.service (name, base_work_cost, is_preven
SELECT
    'Winter Check',
    0,
    true
WHERE NOT EXISTS (
    SELECT 1
    FROM main_schema.service
    WHERE name = 'Winter Check'
);


SELECT * FROM main_schema.service LIMIT 1;
```

Data Output Messages Notifications

	service_id [PK] integer	name character varying (150)	description text	base_work_cost numeric (10,2)	is_preventive boolean
1	1	Winter Check	[null]	0.00	true

Рисунок 11 — Добавленная акция

Команда UPDATE

Цель:

Повысить стоимость всех услуг на 10% для сервисов, которые выполнялись в феврале 2026 года.

Что я сделал:

Использовал `UPDATE` с подзапросом через `EXISTS`, чтобы изменить только те услуги, которые реально использовались в заказах за указанный период.

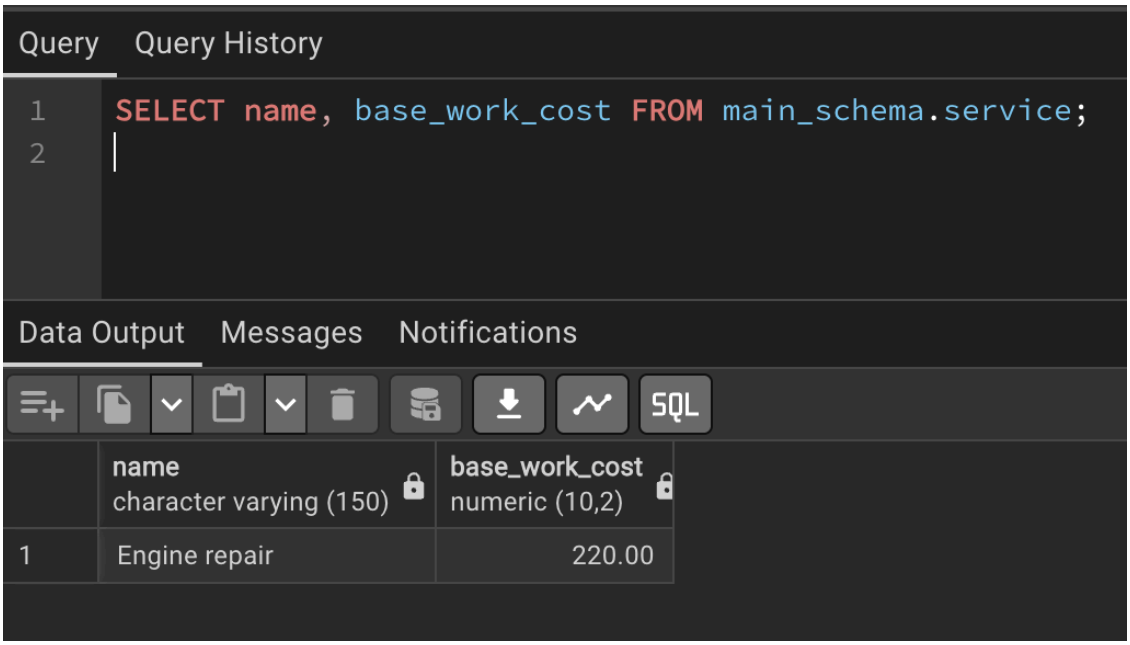


Рисунок 12 — Результаты до команды UPDATE

Query Query History

```sql
UPDATE main_schema.service s
SET base_work_cost = base_work_cost * 1.10
WHERE EXISTS (
    SELECT 1
    FROM main_schema.contract_service cs
    JOIN main_schema.contract ct ON ct.contract_id = cs.contract_id
    WHERE cs.service_id = s.service_id
      AND EXTRACT(MONTH FROM ct.actual_completion_date) = 2
      AND EXTRACT(YEAR FROM ct.actual_completion_date) = 2026
);

SELECT name, base_work_cost FROM main_schema.service;
```

Data Output Messages Notifications

Showin

	name character varying (150)	base_work_cost numeric (10,2)
1	Engine repair	242.00

Рисунок 13 — Использование команды UPDATE

Команда DELETE.

Цель:
Удалить старых клиентов, которые ни разу не завершили ни одного заказа (нет `completed` контрактов)

Что я сделал:
Использовал `DELETE` с подзапросом `NOT EXISTS`, чтобы найти клиентов без завершённых заказов и удалить их из системы.

Query Query History

```
SELECT * FROM main_schema.client;
```

Data Output Messages Notifications

Showing rows: 1 to 2

	client_id [PK] integer	full_name character varying (150)	phone character varying (20)	email character varying (100)	client_address character varying (255)
1	1	Active Client	+111	[null]	[null]
2	2	Inactive Client	+222	[null]	[null]

Рисунок 14 — Результаты ДО использования команды

Query Query History

```
DELETE FROM main_schema.client c
WHERE NOT EXISTS (
    SELECT 1
    FROM main_schema.contract ct
    WHERE ct.client_id = c.client_id
      AND ct.status = 'completed'
);

SELECT * FROM main_schema.client;
```

Data Output Messages Notifications

Showing rows: 1 to 1

	client_id [PK] integer	full_name character varying (150)	phone character varying (20)	email character varying (100)	client_address character varying (255)
1	1	Active Client	+111	[null]	[null]

Рисунок 15 — Использование команды DELETE

Индексы и анализ планов запросов

Простой индекс.

Цель:

Проверить влияние простого индекса на скорость поиска клиента по номеру телефона.

Что сделал:

Смоделировал ситуацию поиска клиента по полю `phone`. Сначала запрос выполняется без индекса (полное сканирование таблицы). Затем создаётся B-Tree индекс и повторяется анализ с использованием `EXPLAIN ANALYZE`.

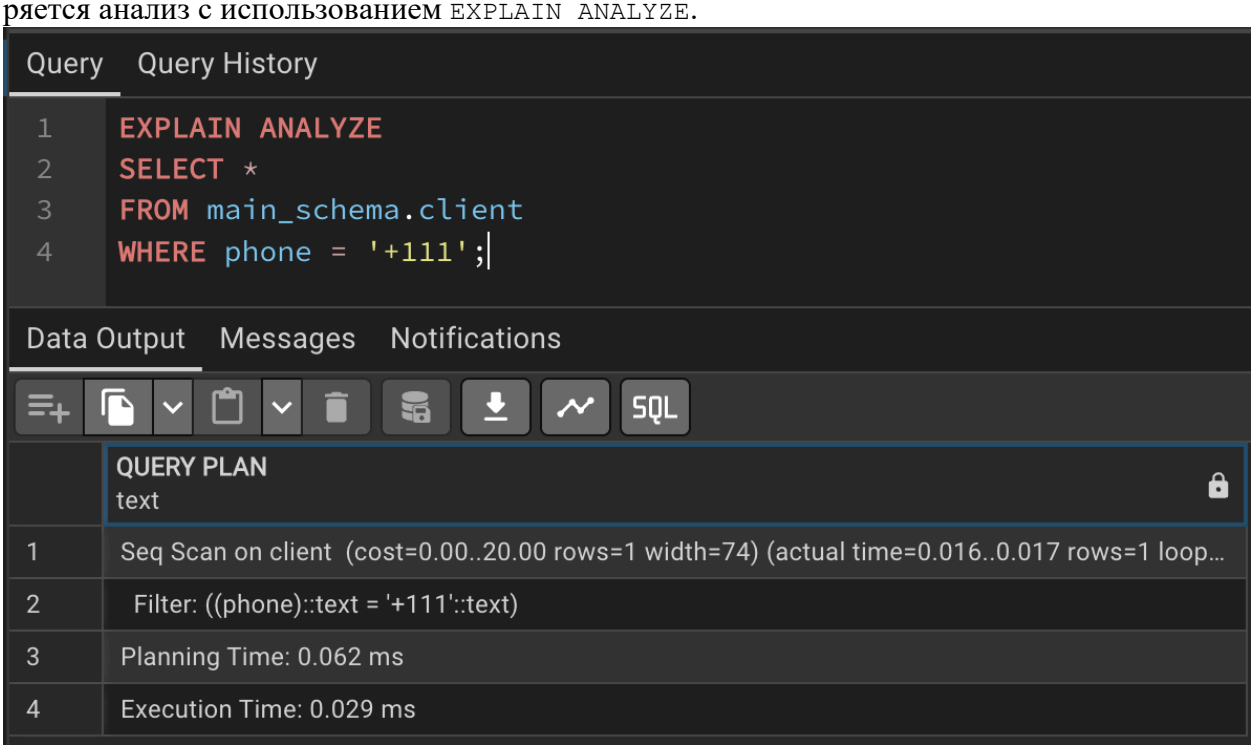


Рисунок 16 — Результаты поиска поиска ДО индекса

Query Query History

```
1
2   CREATE INDEX idx_client_phone
3   ON main_schema.client(phone);
```

Data Output Messages Notifications

```
CREATE INDEX

Query returned successfully in 42 msec.
```

Рисунок 17 —Создания индекса

Query Query History

```
1   EXPLAIN ANALYZE
2   SELECT *
3   FROM main_schema.client
4   WHERE phone = '+111';
5
```

Data Output Messages Notifications

Showing rows: 1 to

	QUERY PLAN text
1	Index Scan using idx_client_phone on client (cost=0.27..8.28 rows=1 width=74) (actual time=0.046..0.047 rows=2 loo…
2	Index Cond: ((phone)::text = '+111'::text)
3	Planning Time: 0.073 ms
4	Execution Time: 0.062 ms

Рисунок 18 — Новые результаты

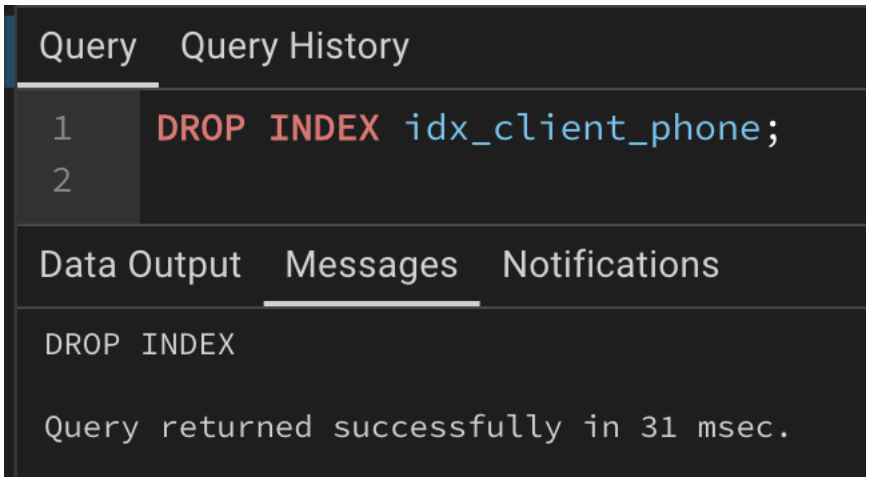


Рисунок 19 —Удаление индекса

Вывод:
После создания индекса план запроса меняется с `Seq Scan` на `Index Scan`, что уменьшает время поиска за счёт прямого обращения к индексированной колонке.

Составной индекс

Цель:

Оценить эффективность составного индекса при фильтрации данных по нескольким полям (марка и год автомобиля).

Что сделал:

Расширил объём данных в таблице `car` и выполнил запрос фильтрации по двум условиям. Сначала без индекса (полное сканирование), затем с составным индексом.

Query Query History

```
EXPLAIN ANALYZE
SELECT *
FROM main_schema.car
WHERE brand = 'Toyota'
  AND production_year = 2018;
```

Data Output Messages Notifications

Showing r

	QUERY PLAN text
1	Seq Scan on car (cost=0.00..2526.00 rows=25027 width=236) (actual time=0.009..17.074 rows=50000 loop...
2	Filter: (((brand)::text = 'Toyota'::text) AND (production_year = 2018))
3	Rows Removed by Filter: 50000
4	Planning Time: 0.649 ms
5	Execution Time: 18.719 ms

Рисунок 20 — Результаты поиска поиска ДО индекса

Scratch Pad Query History

```
CREATE INDEX idx_car_brand_year
ON main_schema.car(brand, production_year);

EXPLAIN ANALYZE
SELECT *
FROM main_schema.car
WHERE brand = 'Toyota'
  AND production_year = 2018;
```

Data Output Messages Notifications

Showing rows: 1 to 7

	QUERY PLAN text
1	Bitmap Heap Scan on car (cost=344.82..1746.22 rows=25027 width=236) (actual time=0.752..3.274 rows=50000 loops=1)
2	Recheck Cond: (((brand)::text = 'Toyota'::text) AND (production_year = 2018))
3	Heap Blocks: exact=1026
4	-> Bitmap Index Scan on idx_car_brand_year (cost=0.00..338.56 rows=25027 width=0) (actual time=0.682..0.682 rows=50000 loo...
5	Index Cond: (((brand)::text = 'Toyota'::text) AND (production_year = 2018))
6	Planning Time: 0.243 ms
7	Execution Time: 4.132 ms

Рисунок 21 —Создания индекса и новые результаты

```
Query   Query History
1   DROP INDEX idx_car_brand_year;
2

Data Output   Messages   Notifications
DROP INDEX

Query returned successfully in 30 msec.
```

Рисунок 22—Удаление индекса

ВЫВОДЫ

В ходе выполнения лабораторной работы были получены практические навыки работы с СУБД PostgreSQL, в частности при использовании модифицирующих операций с применением подзапросов. Было рассмотрено применение вложенных запросов как способа динамического формирования условий для операций INSERT, UPDATE и DELETE.

Также были изучены представления (VIEW), позволяющие формировать удобные виртуальные структуры данных для упрощения анализа информации и повышения удобства работы с отчётами.

Отдельное внимание уделялось анализу планов выполнения запросов с использованием инструмента EXPLAIN ANALYZE. На основе практических экспериментов с увеличенными объёмами данных была подтверждена эффективность использования индексов, в том числе составных, для ускорения выполнения запросов.